

CHECKPOINT_2

Paste everything below into the next chat.

CHECKPOINT_2 — AERIS SYSTEM STATUS AND NEXT DEVELOPMENT PHASE

Project Context

We are developing AERIS, a modular software platform for AI-driven conceptual aircraft design.

The long-term goal of the platform is to:

generate parametric aircraft geometries

evaluate aerodynamic properties

generate large datasets

train ML surrogate models

support multidisciplinary design optimization (MDAO)

The project is currently in early infrastructure and geometry pipeline development.

The software must support:

reproducible experiments

large automated dataset generation

modular geometry generators

integration with AeroSandbox and other solvers

ML training pipelines

The architecture must follow software engineering best practices, not ad-hoc research scripts.

Current Repository Structure

The repository currently follows a Python package + pipeline architecture:

src/aeris/

cli.py

commands/

common/

geometry/

pipeline/

Supporting directories:

configs/

data/

documents/

dev/

tests/

Important directories:

configs/

Contains YAML configuration files defining experiment parameters.

Example:

configs/geometry/wing_bwb.yaml

These configs determine generator type, seed, and outputs.

data/

Stores generated outputs.

Runs are stored in:

data/runs/<run_id>/

Each run contains:

artifacts/

logs/

manifest.json

input_config.yaml

Artifacts include:

geometry_summary.json

control_points.csv

planform_sections.csv

section_3d.csv

planform.png

This structure supports experiment reproducibility.

src/aeris/common/

Infrastructure utilities:

config.py

paths.py

logging_utils.py

Responsibilities:

load YAML configs

create run folders

manage logging

src/aeris/geometry/

Core geometry engine.

Modules include:

planform.py

sections.py

params.py

validation.py

export.py

plotting.py

aerosandbox_adapter.py

Responsibilities:

Module Purpose

planform generate BWB planform geometry

sections build section geometry

params configuration schema

validation geometry validity checks

export export CSV and JSON

plotting visualization

aerosandbox_adapter convert geometry to AeroSandbox objects

src/aeris/pipeline/

Execution pipelines.

Important file:

pipeline/geometry_run.py

Responsibilities:

load config

initialize RNG

generate geometry

build sections

build AeroSandbox model

export artifacts

write manifest

log execution

src/aeris/commands/

CLI commands.

Example:

```
aeris geometry generate
```

This calls the geometry pipeline.

Current Generator Integration

A research script called:

```
Planform_2nd_Approach_V5
```

was successfully integrated into the architecture.

The generator logic was broken into modules:

```
planform.py
```

```
sections.py
```

The generator produces:

BWB planform

section geometry

twist distribution

dihedral distribution

AeroSandbox aircraft

Outputs are exported to CSV and JSON artifacts.

The pipeline now successfully runs:

```
aeris geometry generate --config configs/geometry/wing_bwb.yaml
```

Example output:

AeroSandbox aspect ratio ≈ 2.94

Artifacts are saved correctly.

Current System Capabilities

The system can now:

- ✓ load configs
- ✓ create reproducible run environments
- ✓ generate BWB geometries
- ✓ export geometry datasets
- ✓ generate visualization plots
- ✓ build AeroSandbox aircraft objects
- ✓ log runs and store manifests

The software has moved from standalone scripts → modular architecture.

Current Limitations

The system currently generates one geometry per run.

It does not yet support:

batch dataset generation

generator versioning

automated parameter sweeps

dataset storage structures

ML training integration

These must be addressed next.

Next Development Phase

The next phase is Dataset Generation Infrastructure.

The goals are:

- 1 enable generation of thousands of geometries automatically

- 2 version geometry generators

- 3 create dataset storage format

- 4 prepare metadata for ML training

Tasks for Next Phase

The next chat should help design and implement:

1 — Generator versioning

Example:

bwb_segmented_v1

bwb_segmented_v2

This ensures dataset traceability.

2 — Batch geometry generation command

New CLI command:

aeris dataset generate

Example usage:

aeris dataset generate \

--config configs/geometry/wing_bwb.yaml \

--n 1000

This should generate 1000 geometries with different seeds.

3 — Dataset directory structure

Proposed:

data/datasets/

 bwb_dataset_v1/

 geometry/

 metadata.csv

 configs/

4 — Dataset metadata

Each geometry should store:

seed

generator

span

area

aspect_ratio

section count

twist statistics

dihedral statistics

This becomes the ML dataset label table.

5 — Generator API stabilization

Define a clean generator interface:

```
generate_geometry(config, rng) -> geometry_object
```

This will allow multiple generators to coexist.

What the Next Chat Should Do

The next chat should:

- 1 design dataset generation architecture

- 2 implement dataset CLI command

- 3 implement generator versioning

- 4 design dataset metadata schema

5 update pipeline structure if necessary

The goal is to move the system from:

single geometry pipeline

to

dataset generation engine

which will later feed the ML training pipeline.